

Database Fundamentals with PostgreSQL

1. Scenario — FitZone Gym Management System

You are asked to design and build a complete database for a gym called FitZone, covering the full workflow from member registration to attending sessions and rating them. The database must be built using exactly 6 tables — no more, no less.

The 6 required tables are:

- members — gym members. Contact info and address fields (street, city, country) live directly on this table.
- trainers — each trainer may optionally have a senior Mentor, who is also a trainer (a self-reference within the same table).
- categories — e.g. Yoga, Cardio, Weightlifting, Boxing, Zumba, Pilates.
- classes — each class belongs to exactly one category, and may have more than one Session at different times.
- sessions — each session belongs to one class and is led by one trainer, with a date, start/end time, and room.
- bookings — a member books a session; this table also stores the member's rating (1-5) and comment for that booking once it has taken place. This single table therefore resolves the Many-to-Many relationship between members and sessions, and also plays the role of a feedback record.

Design note: Notice that address fields were folded directly into members instead of a separate addresses table, since every member has exactly one address (1:1) . The same idea was applied to ratings/comments, which were folded into bookings instead of a separate feedback table.

Fixed data that everyone must follow (to keep grading consistent):

- Categories: exactly 5 categories, but only 4 of them currently have classes assigned — the 5th (e.g. Pilates) must have zero classes for now.
- Trainers: at least 6 trainers, with at least one having a Mentor from the same table .
- Classes: 10 classes distributed across the 4 active categories only.
- Sessions: at least 20 sessions distributed across the classes.
- Members: at least 10 members. Exactly one member must have zero bookings.
- Bookings: at least 20 bookings, with some members having more than one booking. At least 15 of these bookings must have a rating filled in (1-5) so the aggregation queries have enough data.

2. Requirements

The requirements are split into phases, in the same order covered in the lectures. Complete each phase in order, and document your work in one organized SQL file with comments explaining each section.

Phase 1: Schema Diagram & Normalization

1. You may use [ERDPlus](#) Tool To draw Schema Diagram showing the 6 real tables, their columns, data types, and key types.

Then write a short write-up explaining:

2. Assume that initially all the system's data lived in one big Unnormalized Table combining member + session + class + trainer + rating data together. Show what this table would look like and what problems it has (data duplication, update anomalies, a repeating/multi-valued ratings column, etc.).

Phase 2: Create the Database and Tables (DDL)

Create a database named `fitzone_db`, and create all 6 tables described above, while following these rules:

- Every table has a clear Primary Key (SERIAL / IDENTITY preferred).
- Every Foreign Key is explicitly defined, including the self-referencing FK on trainers.
- Use NOT NULL on mandatory fields (e.g. email, name, join date).
- Use UNIQUE on member and trainer emails.
- Use a CHECK constraint at least twice (e.g. booking rating between 1 and 5, or member status restricted to specific values like 'active'/'inactive').
- Use DEFAULT at least twice (e.g. booking date defaults to today, or status defaults to 'active').

After creating the tables, run the following commands and record them in the SQL file:

```
-- 1) Add a new column:  
Update the structure of the members table so that it can store loyalty points for each  
member. The new field should be initialized with a default value of 0.  
  
-- 2) Change an existing column's type:  
Modify the members table so that the phone field can store phone numbers with up to 20  
characters.  
  
-- 3) Truncate a table and restart identity:  
Remove all records from the bookings table and reset the auto-incrementing ID values so that future records  
start numbering from the beginning.
```

Note: The TRUNCATE command is executed after data has already been inserted, purely as a learning exercise — afterward, the bookings data must be re-inserted. Make this clear with comments in the SQL file so the grader doesn't think the table is missing data.

Phase 3: Insert Data

Insert realistic, logically consistent data into all 6 tables, meeting the minimum counts listed in the scenario above. Make sure the data actually supports all the required queries later on (i.e. there must be: a member with no bookings, a trainer with a Mentor, a category with zero classes, and so on).

Phase 5: SQL Fundamentals & DML

Run the following queries exactly (write the query and its result for each):

3. Show all members (full name and email) ordered by join date ascending, using AS to alias the full name column.
4. Show the distinct cities found in the members table.
5. Show only the members whose status is 'active'.
6. UPDATE a trainer's years of experience, using RETURNING to return the row after the update.
7. DELETE one dummy booking you inserted specifically for this purpose, using RETURNING to return the deleted row.

Phase 6: Security & Administration (DCL)

Create two users and assign their permissions as follows:

```
-- Create a read-only user:  
Create a new database user named readonly_user with a secure password. The user should be  
able to connect to the fitzone_db database and have read-only access to all tables in the  
public schema.
```

```
-- Create an operations manager user
Create a database user named manager_user with a secure password. Grant the user
permission to view, add, and update records in all tables within the public schema.

-- Revoke update permission from manager_user ( ANY Table ).
```

Write a short paragraph explaining this database administrator working in a company. Include realistic scenarios where permissions are granted and later revoked.

Phase 7: Joins

Run one query for each of the following join types, each answering a clear business question:

8. show every session with its class name and the responsible trainer's name.
9. show all members with their booking count, including members with zero bookings.
10. show all sessions even if they have no bookings (same idea as the previous question, opposite direction).
11. show all categories together with all classes, including the category that currently has zero classes.
12. show each trainer along with the name of their Mentor (if any).

Phase 7: Aggregation , Grouping & Subqueries

13. Count the number of bookings per member.
14. Average rating per trainer, based on the rating column in bookings.
15. Highest and lowest rating given for each class.
16. Total loyalty_points across all members, grouped by city.
17. Use GROUP BY with HAVING to show categories that have more than two classes.
18. Show members who have never made a booking (using NOT IN or NOT EXISTS).
19. Show trainers whose average rating is above the overall average rating of all trainers.

Phase 8: Indexing

Improve the query performance on the **members** table by creating an index on the column used to search for members by their email address.

Improve the query performance on the **bookings** table by creating an index on the column used to identify sessions.

```
-- Then drop one of the indexes as a test
```

3. Notes & Best Practices (Must Follow)

- Table names: plural, lowercase, underscore-separated, e.g. members, not Member or MemberTable.
- Column names: singular and clearly meaningful, e.g. first_name, not fn or Name1.
- Primary key: simply name it id, or member_id if you want extra clarity in tables with many relationships.
- Foreign key: always name it after the referenced table + _id, e.g. member_id, trainer_id.
- Avoid reserved SQL keywords as table or column names (e.g. user, order, table).
- Use appropriate data types: VARCHAR with a sensible length, DATE for dates, BOOLEAN for binary states instead of numbers or text.
- Comment (--) each section of the SQL file so it stays readable and easy to grade.

- Use organized file/folder names when submitting .
- Exactly 6 tables are required — do not add extra tables (e.g. no separate addresses or feedback tables).

4. Deliverables

Submit a ZIP folder named after you, containing:

20. A Schema Diagram file (image or PDF).
 21. One SQL file named `fitzone_assignment.sql` containing all DDL/DML/DCL commands in order, with clear comments.
-